

Linear Regression: Technical Reference Guide

Mathematical Foundations, Assumptions, Optimization, Regularization & Implementation

Prepared for Machine Learning Revision & Advanced Learning

1 Problem Formulation & Output Type

Linear Regression is a foundational supervised machine learning algorithm designed to model the mathematical relationship between a continuous scalar target variable $y \in \mathbb{R}$ and one or more independent predictor variables $\mathbf{x} \in \mathbb{R}^k$.

- **Supervised Learning Domain:** Operates on labeled training datasets $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$.
- **Output Characterization:** Continuous real-valued predictions (e.g., real estate pricing, revenue projection, temperature estimation).
- **Algorithmic Taxonomy:**
 - **Simple (Univariate) Linear Regression:** Models target y using a single feature x .
 - **Multiple (Multivariate) Linear Regression:** Models target y using k features $(x_1, x_2, \dots, x_k)$.

Key Practical Use Cases

- **Real Estate Valuation:** Predicting house prices based on physical square footage, bedrooms, and location metrics.
- **Sales & Demand Forecasting:** Estimating retail sales volumes based on promotional spend and seasonal indices.
- **Financial Analysis:** Evaluating asset betas, risk-adjusted returns, and macroeconomic sensitivity.
- **Healthcare & Clinical Research:** Modeling disease progression markers against patient demographic profiles.

2 Hypothesis Function & Objective Optimization

2.1 Hypothesis Equation

For a feature vector with k dimensions, the hypothesis function $\hat{y} = h_{\theta}(\mathbf{x})$ is defined mathematically as:

$$\hat{y} = h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_k x_k = \sum_{j=0}^k \theta_j x_j = \boldsymbol{\theta}^T \mathbf{x} \quad (1)$$

where $\boldsymbol{\theta} = [\theta_0, \theta_1, \dots, \theta_k]^T$ denotes the parameter vector (bias/intercept θ_0 and weight coefficients $\theta_{1..k}$), and $\mathbf{x} = [1, x_1, \dots, x_k]^T$ is the augmented feature vector.

2.2 Objective Function: Mean Squared Error (MSE)

To quantify prediction error, Linear Regression minimizes the Mean Squared Error (MSE) cost function $J(\boldsymbol{\theta})$ across n training instances:

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \left(\hat{y}^{(i)} - y^{(i)} \right)^2 = \frac{1}{n} \sum_{i=1}^n \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2 \quad (2)$$

2.3 Optimization Method 1: Ordinary Least Squares (OLS) Analytical Solution

Setting the vector gradient $\nabla_{\theta} J(\theta) = \mathbf{0}$ yields the closed-form analytical solution known as the **Normal Equation**:

$$\theta^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (3)$$

Note: Matrix inversion $(\mathbf{X}^T \mathbf{X})^{-1}$ incurs a computational complexity of $\mathcal{O}(k^3)$, making analytical OLS computationally expensive for large feature counts ($k > 10,000$) or singular matrices (when features exhibit perfect multicollinearity).

2.4 Optimization Method 2: Gradient Descent Iterative Solver

For high-dimensional or massive datasets, **Gradient Descent** iteratively updates parameter values in the direction of steepest loss decrease:

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) = \theta_j - \alpha \frac{2}{n} \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} \quad (4)$$

where $\alpha > 0$ represents the learning rate hyperparameter.

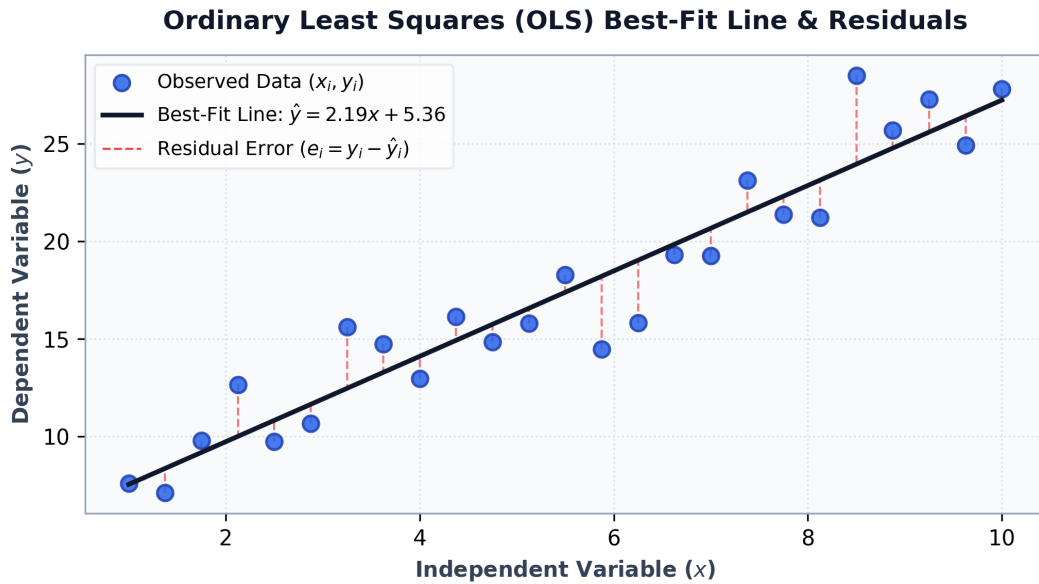


Figure 1: Visualization of Ordinary Least Squares (OLS) Best-Fit Line fitting observations while minimizing vertical residual distances ($e_i = y_i - \hat{y}_i$).

3 Core Statistical Assumptions

The statistical validity, hypothesis testing reliability, and theoretical optimality (Gauss-Markov Theorem) of Linear Regression rely on seven core assumptions:

Assumption	Mathematical Definition / Condition	Diagnostic & Remedy
1. Linearity	$E[y \mathbf{X}] = \mathbf{X}\boldsymbol{\theta}$ (Linear combination of parameters)	Residual vs. fitted plot, scatter plots.
2. Error Independence	$\text{Cov}(\epsilon_i, \epsilon_j) = 0, \forall i \neq j$	Durbin-Watson test statistic (1.5 – 2.5).
3. Homoscedasticity	$\text{Var}(\epsilon_i \mathbf{X}) = \sigma^2$ (Constant error variance)	Breusch-Pagan test, Box-Cox target transform.
4. Normality of Errors	$\epsilon_i \sim \mathcal{N}(0, \sigma^2)$	Q-Q plot, Shapiro-Wilk statistical test.
5. No Multicollinearity	$\text{rank}(\mathbf{X}) = k + 1$ (Predictors are linearly independent)	Variance Inflation Factor (VIF < 5).
6. No Autocorrelation	Errors exhibit no repeating patterns across time	Time series lag plot, autoregressive models.
7. Additivity	Combined effect of predictors is additive	Add feature interaction terms ($x_i \cdot x_j$).

Table 1: Summary of standard Gauss-Markov and Classical Linear Model assumptions.

4 Advantages & Limitations (Pros & Cons)

Trade-off Matrix

- Advantages (Pros):**
- **High Interpretability:** Each coefficient θ_j directly quantifies the marginal change in target y per unit increase in x_j , holding all other features constant.
 - **Computational Efficiency:** Extremely fast to train and run inference; low computational and memory overhead.
 - **Guaranteed Global Optimum:** The MSE loss surface is strictly convex, guaranteeing convergence to a unique global minimum without local minima traps.
 - **Standard Baseline:** Provides a highly reliable baseline model for regression benchmarks.
- Limitations (Cons):**
- **Linearity Assumption:** Incapable of modeling non-linear relationships without explicit feature transformations or polynomial expansions.
 - **Outlier Sensitivity:** The squared loss penalty excessively weights extreme outliers, skewing slope and intercept estimations.
 - **Multicollinearity Vulnerability:** Highly correlated input features cause coefficient instability and standard error inflation.

5 Regularization Techniques & Hyperparameters

Regularization prevents model overfitting by adding a parameter complexity penalty to the MSE objective.

5.1 Regularization Formulations

- **L1 Regularization (Lasso Regression):** Adds an L1-norm penalty to drive uninformative feature coefficients strictly to zero, performing implicit feature selection:

$$J_{\text{Lasso}}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \left(\hat{y}^{(i)} - y^{(i)} \right)^2 + \lambda \sum_{j=1}^k |\theta_j| \quad (5)$$

- **L2 Regularization (Ridge Regression):** Adds an L2-norm penalty to shrink feature weights toward zero without eliminating them, stabilizing estimation under multicollinearity:

$$J_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \left(\hat{y}^{(i)} - y^{(i)} \right)^2 + \lambda \sum_{j=1}^k \theta_j^2 \quad (6)$$

- **Elastic Net Regression:** Blends L1 and L2 penalties using a mixing ratio hyperparameter $r \in [0, 1]$:

$$J_{\text{ElasticNet}}(\boldsymbol{\theta}) = \text{MSE} + r\lambda \sum_{j=1}^k |\theta_j| + \frac{1-r}{2}\lambda \sum_{j=1}^k \theta_j^2 \quad (7)$$

5.2 Key Hyperparameters

- λ or α (alpha): Regularization penalty multiplier controlling model complexity penalty.
- r (l1_ratio): ElasticNet mixing ratio between L1 ($r = 1$) and L2 ($r = 0$) norms.
- `fit_intercept`: Boolean flag determining whether to calculate bias intercept θ_0 .

6 Performance Evaluation Metrics

Metric	Mathematical Formula	Interpretation & Key Characteristics
Mean Absolute Error (MAE)	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Average absolute magnitude of errors in target units. Robust to outliers.
Mean Squared Error (MSE)	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Differentiable loss function; heavily penalizes larger errors.
Root Mean Squared Error (RMSE)	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	Square root of MSE; interprets error back in original target units.
R-Squared (R^2)	$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	Proportion of total target variance explained by the model ($[0, 1]$).
Adjusted R^2	$R^2_{adj} = 1 - \left[\frac{(1-R^2)(n-1)}{n-k-1} \right]$	Adjusts R^2 for the number of predictors k ; penalizes irrelevant features.

Table 2: Standard statistical evaluation metrics for regression models.

7 End-to-End Executable Python Code Implementation

```

1 import numpy as np
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import Ridge
4 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
5
6 # 1. Synthetic Dataset Generation
7 np.random.seed(42)
8 X = np.random.rand(200, 3) * 10 # 200 samples, 3 features
9 y = 3.5 * X[:, 0] - 2.0 * X[:, 1] + 1.2 * X[:, 2] + 5.0 + np.random.normal(0, 1.5,
10     size=200)
11
12 # 2. Train-Test Split
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state
14     =42)
15
16 # 3. Model Training (Ridge Regression)
17 model = Ridge(alpha=1.0)
18 model.fit(X_train, y_train)
19
20 # 4. Inference
21 y_pred = model.predict(X_test)
22
23 # 5. Metrics Evaluation
24 mse = mean_squared_error(y_test, y_pred)
25 rmse = np.sqrt(mse)
26 mae = mean_absolute_error(y_test, y_pred)
27 r2 = r2_score(y_test, y_pred)
28
29 print(f"Intercept (b): {model.intercept_:.4f}")
30 print(f"Coefficients (m): {model.coef_}")
31 print(f"MAE: {mae:.4f} | RMSE: {rmse:.4f} | R2 Score: {r2:.4f}")

```

Listing 1: Complete Scikit-Learn Pipeline for Linear Regression with Ridge Regularization and Performance Evaluation.